



เอกสารสรุปสถาปัตยกรรมระบบ & คู่มือ สัมภาษณ์นักพัฒนา

ระบบบริการหลังการขายศูนย์ซ่อมบำรุงรถยนต์ (Aftersales Service
Dashboard)

เอกสารฉบับนี้จัดทำขึ้นเพื่อให้ทำความเข้าใจโครงสร้างการทำงาน เทคโนโลยีที่ใช้งาน
และหลักการออกแบบระบบอย่างละเอียด รวมถึงคู่มือคำถาม-ตอบสัมภาษณ์สำหรับคัด
เลือกนักพัฒนาซอฟต์แวร์โดย Senior Programmer

บทบาทผู้จัดทำ: Senior Programmer / Systems Architect

เทคโนโลยีหลัก: Next.js 16 (React 19) | Spring Boot 3.3.5 | MySQL

วันที่อัปเดตล่าสุด: 9 มิถุนายน 2569

ส่วนที่ 1: รายละเอียดของ Webapp อย่างละเอียด (System Overview & Architecture)

ระบบ **Aftersales Service Dashboard** คือเว็บแอปพลิเคชันเกรดพรีเมียมที่ได้รับการออกแบบมาเป็นพิเศษสำหรับ ศูนย์บริการซ่อมบำรุงรถยนต์ เพื่อใช้ในการจัดการและติดตามสถานะงานซ่อมแบบเรียลไทม์ ตั้งแต่ขั้นตอนการนำรถเข้าศูนย์ (Vehicle Intake), การตรวจประเมินอาการ (Triage/Diagnostics), การจัดสรรทีมงานช่างเทคนิค (Mechanic Assignment), การเบิกจ่ายอะไหล่ (Spare Parts Allocation) ไปจนถึงการชำระเงินและการส่งมอบรถคืนให้กับลูกค้า (Delivery)

1. โครงสร้างสถาปัตยกรรมของระบบ (System Architecture)

ระบบนี้ใช้สถาปัตยกรรมแบบ **Client-Server Separation** แยกส่วน Frontend และ Backend อย่างเด็ดขาดผ่าน RESTful APIs ช่วยให้การพัฒนา บำรุงรักษา และการสเกลระบบทำได้ง่ายและยืดหยุ่นสูง:

องค์ประกอบ	เทคโนโลยี (Tech Stack)	รายละเอียดและหน้าที่หลัก
Frontend (หน้าบ้าน)	Next.js 16 (App Router) React 19 TypeScript Vanilla CSS	ระบบหน้าเว็บแอปพลิเคชันแบบ Single Page Application (SPA) ใช้ Next.js App Router และ React 19 ในการเรนเดอร์ออร์บอร์ต Kanban และ Modal ต่างๆ มีการเชื่อมต่อแบบ Client-side Component และมี Fallback Simulation คอยดักกรณี Backend ชัดข้อง
Backend (หลังบ้าน)	Spring Boot 3.3.5 Spring Data JPA Spring Security Java 17	ระบบ API Gateway และ Business Logic บริหารจัดการสิทธิ์ผู้ใช้ (RBAC) การตรวจสอบสิทธิ์การเปลี่ยนสถานะใบงานซ่อม (State-Machine) และการประมวลผลทางการเงิน (ค่าแรง + ค่าอะไหล่ + VAT 7%)
Database (ฐานข้อมูล)	MySQL Hibernate	ฐานข้อมูลเชิงสัมพันธ์ เก็บข้อมูลความสัมพันธ์ของ Users (พนักงาน), Customers (ลูกค้า), Vehicles (รถยนต์), SpareParts (ชิ้นส่วนอะไหล่), ServiceTickets (ใบงาน), และ TicketItems (อะไหล่ที่ใช้ในแต่ละใบงาน)

2. ฟีเจอร์การทำงานหลักของระบบ (Core Features)

- **Interactive Kanban Board:** ฝั่งงานขั้นตอนการทำซ่อมบำรุง 5 ขั้นตอน ได้แก่ **รับรถ / ตรวจอาการ (PENDING)** → **กำลังดำเนินการซ่อม (IN_PROGRESS)** → **รอชิ้นส่วนอะไหล่ (WAITING_FOR_PARTS)** → **ซ่อมเสร็จสิ้น (COMPLETED)** → **ส่งมอบรถ / ปิดงาน (DELIVERED)**
- **Role-Based Access Control (RBAC):** การควบคุมการเข้าถึงตามบทบาทการทำงานของพนักงานซ่อมบำรุง:
 - *Admin:* ทำการจัดการได้ทุกอย่าง รวมถึงการลบใบงาน (Soft-delete)
 - *Service Advisor (พนักงานรับรถ):* เปิดใบงานซ่อม มอบหมายงานช่าง จัดการบิล อะไหล่ แต่ไม่สามารถลบใบงานได้
 - *Mechanic (ช่างเทคนิค):* ดูรายละเอียดใบงาน และอัปเดตสถานะบอร์ดได้เฉพาะขั้นตอนซ่อมบำรุงเท่านั้น (ไม่เห็นบิล/ราคาอะไหล่)
- **Smart Spare Parts Allocation (การจัดการชิ้นส่วนอะไหล่แบบรวมจำนวน):** เมื่อแอดมินหรือพนักงานรับรถเพิ่มชิ้นส่วนอะไหล่ตัวเดิมซ้ำลงในใบงานเดียวกัน ระบบหลังบ้านของ Spring Boot จะทำการอัปเดตจำนวน (Merge Quantity) โดยอัตโนมัติ แทนที่จะสร้างแถวข้อมูลใหม่ขึ้นมาซ้ำซ้อน ซึ่งป้องกันปัญหา Database Unique Constraint Violation ได้อย่างสมบูรณ์แบบ
- **Soft-Delete & Inventory Auto-Restore:** การลบใบงานอย่างปลอดภัยโดยเปลี่ยนสถานะ (Soft-delete) และเมื่อใบงานถูกลบ ระบบจะคืนจำนวนสต็อกอะไหล่ชิ้นนั้นๆ กลับเข้าคลังสินค้าโดยอัตโนมัติเพื่อป้องกันข้อมูลสต็อกเพี้ยน
- **Styled Excel Export:** ความสามารถในการส่งออกสรุยอดใบงานบริการหลังการขายทั้งหมดออกมาเป็นไฟล์ Excel โดยเขียนโปรแกรมสร้างไฟล์ XML Spreadsheet เพื่อให้ได้ความสวยงาม ตารางสรุปที่มีการคำนวณยอดเงินรวม (Grand Total) และการระบุสีสันให้ผู้บริหารนำไปประชุมเสนอต่อได้ทันที

ส่วนที่ 2: คำถาม-คำตอบสัมภาษณ์โดย Senior Programmer (Interview Questions & Answers)

คำถามชุดนี้ออกแบบมาเพื่อประเมินความสามารถเชิงลึกของนักพัฒนา (ระดับ Mid ถึง Senior) โดยใช้โค้ดจริงและการแก้ปัญหาจากระบบ **Aftersales Service Dashboard** นี้เป็นกรณีศึกษาหลัก

หมวดที่ 1: Architectural & System Design (การออกแบบระบบและสถาปัตยกรรม)

Q1: ในระบบนี้มีการเลือกแยกส่วน Frontend (Next.js) และ Backend (Spring Boot) ออกจากกันอย่างเด็ดขาด ข้อดีและข้อเสียของการเลือกโครงสร้างรูปแบบนี้มีอะไรบ้าง และในฐานะ Senior คุณจะเลือกใช้แบบใดในโปรเจกต์ขนาดใหญ่?

คำตอบของผู้สมัครที่คาดหวัง:

ข้อดี: 1) Separation of Concerns: ฝั่งหน้าบ้านโฟกัสเรื่อง UX/UI และการแสดงผล ส่วนหลังบ้านโฟกัสเรื่อง Business Logic ความปลอดภัย และข้อมูล 2) Independent Deployment: สามารถแยก Deploy ฝั่งหน้าบ้าน (เช่น บน Vercel) และหลังบ้าน (เช่น บน Railway/AWS) ทำให้แอปโหลดเร็วขึ้นและลดภาระเซิร์ฟเวอร์ 3) Technology Flexibility: สามารถพัฒนาโปรเจกต์อื่น (เช่น Mobile App) มาเชื่อมต่อกับหลังบ้านเดิมได้ง่ายโดยไม่ต้องเขียน API ใหม่

ข้อเสีย: เพิ่มความซับซ้อนในการจัดการ CORS, การจัดการ JWT Authentication ที่ปลอดภัยระหว่างโดเมน และการจัดการ State ให้ตรงกันทั้งสองฝั่ง

มุมมอง Senior: สำหรับโปรเจกต์ขนาดใหญ่ที่มีการทำงานร่วมกันหลายทีม การแยก Frontend/Backend คือแนวทางที่ดีที่สุดเพราะทำให้สเกลงานแยกกันได้ชัดเจน ไม่เกิดคอขวดตอนคอมไพล์โค้ด

Q2: อธิบายความสัมพันธ์ในรูปแบบ Entity-Relationship ระหว่าง ServiceTicket, TicketItem และ SparePart ทำไมเราถึงต้องมี Entity ตรงกลางที่ชื่อว่า TicketItem และความสัมพันธ์นั้นเป็นอย่างไร?

คำตอบของผู้สมัครที่คาดหวัง:

- ความสัมพันธ์ระหว่าง `ServiceTicket` และ `SparePart` เป็นแบบ **Many-to-Many** เนื่องจากใบงาน 1 ใบสามารถใช้อะไหล่ได้หลายชนิด และอะไหล่ 1 ชนิดสามารถนำไปใช้ในหลายใบงานได้

- เราจึงต้องมี Entity ตรงกลางคือ `TicketItem` (หรือ OrderLine) ทำหน้าที่เป็น Join Table เพื่อแปลงความสัมพันธ์ Many-to-Many ให้เป็น **One-to-Many** สองชุด

- สาเหตุที่ต้องมี `TicketItem` เป็น Entity แยกออกมา เนื่องจากเราต้องเก็บข้อมูลคุณสมบัติเพิ่มเติมในความสัมพันธ์นั้น เช่น จำนวนที่เบิกใช้ (**Quantity**), ราคารวมย่อย (**Subtotal**) และ ราคาขาย ณ ขณะนั้น (**Price per unit**) ซึ่งไม่สามารถนำไปเก็บไว้ที่ `ServiceTicket` หรือ `SparePart` โดยตรงได้

Q3: ในการออกแบบบอร์ดงานซ่อม Kanban จะต้องมีการทำ State Machine Validation (เช่น ห้ามข้ามขั้นตอนจากรับรถ (PENDING) ไปส่งมอบรถ (DELIVERED) โดยทันที) คุณคิดว่าควรทำ Validation นี้ที่ฝั่ง Frontend หรือ Backend และมีวิธีการรับประกันความปลอดภัยของสถานะอย่างไร?

คำตอบของผู้สมัครที่คาดหวัง:

- ต้องทำ Validation ที่ฝั่ง Backend เสมอ เนื่องจาก Frontend เป็นสิ่งที่มีโอกาสถูกดัดแปลงหรือบายพาสได้ง่าย (เช่น ผู้ใช้กดปิดสิทธิ์การใช้งานผ่าน Console หรือยิง API Request ตรงโดยไม่ใช้ปุ่มลากบอร์ด)
- ในฝั่ง Frontend (Next.js) เราทำ Validation ในระดับของ UI/UX เท่านั้น เพื่อไม่ให้ผู้ใช้ลากบอร์ดผิดช่องและแสดงแถบสีเตือนความไม่สอดคล้อง
- ในระดับ Backend เราจะใช้ `enum` ในการกำหนดสถานะ และเขียนโค้ดตรวจสอบเงื่อนไข เช่น ในตอนที่รับคำขอ PATCH อัปเดตสถานะของ Ticket ต้องมีเงื่อนไข Switch-case คอยตรวจสอบว่าสถานะใหม่สัมพันธ์กับสถานะเดิมตามเงื่อนไขที่กฎธุรกิจยอมรับหรือไม่ ถ้าผิดพลาดให้โยนข้อความแจ้งเตือนความล้มเหลว (400 Bad Request) กลับไป

หมวดที่ 2: Database & Transaction Management (การจัดการฐานข้อมูลและทรานแซกชัน)

Q4: ในระบบนี้มีการแก้ปัญหา duplicate spare parts ในระดับหลังบ้านด้วยการทำ Merge Quantity ใน `addTicketItem()` ช่วยอธิบายเชิงเทคนิคว่าหากเราปล่อยให้มียอะไหล่ตัวเดียวกัน ซ้ำในตั๋วใบเดียวกันในระดับฐานข้อมูล จะส่งผลเสียอย่างไร?

คำตอบของผู้สมัครที่คาดหวัง:

- หากปล่อยให้ข้อมูลซ้ำซ้อนในตาราง Join Table (`ticket_item`) จะทำให้เกิดปัญหา: 1) **การขัดแย้งของข้อจำกัดฐานข้อมูล (Database Unique Key Constraint Violation):** การออกแบบคีย์ร่วมกันระหว่าง `(ticket_id, part_id)` เป็น Primary/Unique Key จะแจ้ง Error ทันทีเมื่อมีการเพิ่มแถวซ้ำ 2) **การคำนวณผิดพลาด:** ในกรณีที่ไม่ได้ตั้ง Unique Key แต่อะไหล่ถูกแบ่งเป็นสองแถว จะเกิดการอ่านค่าซ้ำซ้อน (Data redundancy) และการจัดการยอดรวมยอดตอนอัปเดตราคาอะไหล่ทำได้ยากขึ้น - การเขียนโค้ดใน `addTicketItem()` เพื่อเช็ก่อนว่าอะไหล่ ID นี้มีอยู่หรือยัง ถ้ามีอยู่แล้วให้ทำการบวกค่า `'quantity'` เพิ่มและอัปเดตแถวข้อมูลเดิม จึงเป็นวิธีที่สะอาดที่สุดและป้องกัน Database Crash ได้อย่างปลอดภัย

Q5: อธิบายฟังก์ชัน Soft-Delete ใน Service Ticket และกระบวนการคืนอะไหล่เข้าสต็อกสินค้า (Inventory Auto-Restore) หลังการลบใบงาน เราจะรับประกันได้อย่างไรว่าข้อมูลทั้งสองตาราง (Ticket และ Spare Part) จะเกิดความถูกต้องเสมอ ไม่เกิดสต็อกค้างชั่วคราว?

คำตอบของผู้สมัครที่คาดหวัง:

- **Soft-Delete:** แทนที่จะใช้คำสั่ง SQL `'DELETE'` ลบแถวข้อมูลออกจากฐานข้อมูลโดยตรง เราจะเพิ่มฟิลด์เช่น `'is_deleted'` หรือการย้ายข้อมูลสถานะใบงาน เพื่อเก็บรักษาข้อมูลอ้างอิงของประวัติงานเอาไว้ตรวจสอบย้อนหลังได้

- **การคืนสต็อก:** เมื่อผู้ใช้กดลบใบงาน ระบบจะวนลูปดึงข้อมูลรายการอะไหล่ (`'TicketItem'`) ทั้งหมดในใบงานนั้น แล้วเพิ่มจำนวนสต็อก (`'stockQuantity'`) กลับเข้าไปในตาราง `'spare_part'` ตามจำนวนที่ใช้ไป

- **การรับประกันความถูกต้อง:** เราต้องกำกับเมธอดลบใบงานใน Spring Boot Service ด้วยแอนโนเทชัน `'@Transactional'` ของ Spring Framework เพื่อยืนยันว่าการเปลี่ยนสถานะใบงานซ่อมและการเพิ่มจำนวนอะไหล่กลับเข้าสต็อก จะต้องเกิดขึ้นพร้อมกันอย่างสมบูรณ์แบบ (Atomicity) หากมีขั้นตอนใดล้มเหลว (เช่น ฐานข้อมูลขัดข้องตอนอัปเดตอะไหล่ชิ้นสุดท้าย) การเปลี่ยนแปลงทั้งหมดจะถูก Rollback กลับทันทีเพื่อรักษาความสอดคล้องของข้อมูล (Data Consistency)

Q6: หากระบบนี้มีการขยายขนาดและมีผู้ใช้งานจองอะไหล่ซ้ำกันพร้อมกันจำนวนมาก (High Concurrency) อาจเกิดปัญหาสต็อกติดลบ หรือการหักสต็อกชนกัน (Race Condition) คุณมีวิธีแก้ไขปัญหานี้ในระดับ Database และ Spring Boot อย่างไร?

คำตอบของผู้สมัครที่คาดหวัง:

มีวิธีแก้ไขยอดนิยม 2 รูปแบบ:

- 1) **Pessimistic Locking (การล็อกแบบระมัดระวัง):** ใช้คำสั่ง SQL ``SELECT ... FOR UPDATE`` ผ่าน JPA ``@Lock(LockModeType.PESSIMISTIC_WRITE)`` เพื่อล็อกแถวข้อมูลของอะไหล่ชิ้นนั้นไว้ไม่ให้เรดอื่นแก้ไขได้ชั่วคราว จนกว่า Transaction ปัจจุบันจะทำงานเสร็จสิ้น เหมาะสำหรับกรณีที่มีการแข่งขันจองสูงมาก
- 2) **Optimistic Locking (การล็อกแบบมองโลกในแง่ดี):** เพิ่มคอลัมน์ ``@Version`` ในระดับ Entity อะไหล่ เมื่อทำการบันทึกข้อมูล JPA จะเช็คเวอร์ชันของข้อมูลที่จะอัปเดตตรงกับปัจจุบันหรือไม่ หากมีเรดอื่นอัปเดตตัดหน้าไปก่อนและเวอร์ชันเปลี่ยน ระบบจะโยน ``OptimisticLockingFailureException`` ออกมาเพื่อให้ระบบของฝั่งเราสั่งให้ลองทำรายการใหม่อีกครั้ง (Retry)
- 3) **Database Constraint:** ใส่เงื่อนไขใน MySQL เช่น ``CHECK (stock_quantity >= 0)`` เพื่อใช้ฐานข้อมูลเป็นแนวรับด้านสุดท้ายกันไม่ให้เกิดสต็อกติดลบเด็ดขาด

หมวดที่ 3: Frontend & Next.js Development (การพัฒนาฝั่งหน้าบ้าน)

Q7: ในโค้ดหน้าบ้านมีการพัฒนา Local Fallback Mechanism เพื่อช่วยให้แอปพลิเคชันยังคงรันตัวอย่างบอร์ดซ่อมรถและส่วนต่างๆ ได้แม้ไม่รัน Backend มีการจัดลำดับการทำงานและข้อควรระวังในเรื่อง ID Mismatch อย่างไร?

คำตอบของผู้สมัครที่คาดหวัง:

- **การจัดลำดับการทำงาน:** เมื่อเว็บเพจโหลด ตัวระบบจะยิง API ไปยังหลังบ้าน หากพบข้อผิดพลาดหรือยิงไม่เข้า (Connection refused) โค้ดจะดักจับด้วย `catch` และนำข้อมูลตั้งต้นจากโมเดลจำลอง `INITIAL\_TICKETS`, `INITIAL\_PARTS\_CATALOG`, และ `MOCK\_MECHANICS` มาเซตลงใน local state แทน
- **การระวังเรื่อง ID Mismatch:** หาก IDs ของข้อมูลทดสอบใน Mock Data (เช่น อะไหล่หมายเลข 201) ไม่ตรงกับข้อมูล Auto-Increment ID ของหลังบ้านจริง (ที่เริ่มจาก ID 1-6) เมื่อระบบหลังบ้านตื่นขึ้นมาทำงาน แล้วฝั่งหน้าบ้านทำการ POST ส่งข้อมูล Item ID 201 ไป จะทำให้เซิร์ฟเวอร์หาข้อมูลไม่เจอและโยน Exception เกิดระบบพัง ดังนั้นการแก้ปัญหาทำได้โดยการจัดเรียง IDs ของ Mock ในหน้าบ้านให้เริ่มต้นที่ 1-6 ตรงตามโครงสร้างฐานข้อมูลหลังบ้าน

Q8: ทำไมเราถึงต้องใช้ `useMemo` ในการคำนวณสรุปผลยอดตัวเลขต่างๆ (Metrics เช่น ยอดรายได้รวม ยอดใบงานรอกกลางซ่อม ยอดอะไหล่รักษาสต็อก) แทนที่จะคำนวณในขั้นตอนการเรนเดอร์ปกติของ React?

คำตอบของผู้สมัครที่คาดหวัง:

- React Component จะทำการ Re-render ใหม่ในทุกๆ ครั้งที่มีการเปลี่ยนแปลงของ State ตัวใดตัวหนึ่งหรือ Parent เปลี่ยนแปลง
- หากเราเขียนฟังก์ชันการคำนวณหาค่าเฉลี่ยหรือสรุปยอดเงินสดรวมลงไปใน การเรนเดอร์ตรงๆ ฟังก์ชันเหล่านั้นจะถูกคำนวณใหม่ทุกรอบการ Re-render แม้ว่าข้อมูลตัวงานซ่อมจะไม่มีการเพิ่มหรือเปลี่ยนแปลงเลยก็ตาม ส่งผลให้เสียประสิทธิภาพการประมวลผล (Performance waste)
- การครอบด้วย `useMemo` และระบุ Dependency array เป็น `[tickets, partsCatalog]` จะบอก React ให้บันทึกความจำ (Cache) ผลลัพธ์นี้ไว้ และจะประมวลผลคำนวณซ้ำใหม่อีกครั้งต่อเมื่อตัวแปร `tickets` หรือ `partsCatalog` มีค่าเปลี่ยนไปเท่านั้น ทำให้เกิดความสิ้นเปลืองในหน้าจอ

Q9: การจัดการกับ Dark Mode ใน Next.js โดยใช้ CSS Variables ร่วมกับ

`document.documentElement.classList.add('dark')` มีกระบวนการอย่างไร และทำไมวิธีนี้จึงมีข้อดีกว่าการเปลี่ยนตัวแปร Theme ในระดับ Component State ทั่วไป?

คำตอบของผู้สมัครที่คาดหวัง:

- กระบวนการ: เมื่อธีมเปลี่ยนเป็น `'dark'` ตัวแอปจะเรียกคำสั่งใส่คลาส `.dark` ไว้ที่ Root Node (`<html>`) และใช้ CSS Variables ในสไตลชีทเพื่อแมปสีข้ามทั้งหมดตามคลาสนั้น
- ข้อดี: 1) ประสิทธิภาพสูง: CSS Variables ช่วยลดรอบการ Re-render ของ React เพราะตัวสไตลชีทยึดหยุ่นที่ระดับเบรวเซอร์ ไม่จำเป็นต้องส่ง Context สีส้มผ่านโปรเจกต์ React ทั่วประเทศ 2) สะดวกและสอดคล้อง: ทำงานร่วมกับ CSS ปกติหรือ TailwindCSS ได้ง่ายมาก และเปลี่ยนธีมให้องค์ประกอบที่รันอยู่นอกโครงสร้างหลักได้รวดเร็ว

หมวดที่ 4: Backend & Security Development (หลังบ้านและความปลอดภัย)

Q10: อธิบายการประยุกต์ใช้งานสิทธิ์ผู้ใช้ (RBAC) ใน Spring Security สำหรับแอปพลิเคชันนี้ หากมีผู้ใช้งานพยายามข้ามขีดจำกัดสิทธิ์ของตนเอง (เช่น ข้างซ่อมรถเทคนิค พยายามส่งคำขอ DELETE ลบตัวซ่อมผ่าน Postman) เราจะป้องกันในระดับ API ได้อย่างไร?

คำตอบของผู้สมัครที่คาดหวัง:

- เราสามารถจำกัดสิทธิ์ในระดับหลังบ้านได้โดยการตั้งค่า ****Method-level Security**** ใน Spring Boot Controller ด้วยแอนโนเทชัน `@PreAuthorize("hasRole('ADMIN')")` บน API ลบใบงาน หรือจำกัดผ่านสถาปัตยกรรมตัวกรอง Security Filter Chain ตัวอย่างเช่น:
`.requestMatchers(HttpMethod.DELETE, "/api/tickets/**").hasRole("ADMIN")`
- วิธีการนี้ทำให้แม้ผู้ใช้งานรู้ URL Endpoint และยิงคำขอลบใบงานเข้ามาโดยตรงด้วยสิทธิ์การตรวจสอบ JWT Token ของพนักงานช่างทั่วไป ระบบสปริงซีเคียวริตี้จะตรวจเช็ค Roles ภายใน Payload ของ Token นั้น และโยนข้อผิดพลาด `403 Forbidden` ปิดกั้นการข้ามสิทธิ์ผู้ใช้ (Privilege Escalation) ในทันที

Q11: ใน `DatabaseSeeder.java` มีการเขียนโค้ดเพื่อแก้ปัญหากรณีฐานข้อมูลเดิมยังมีชื่อยี่ห้อและรุ่นรถยนต์ที่ไม่เป็นไปตามกำหนด (เช่น ยี่ห้ออื่นๆ ที่ไม่ใช่ Isuzu) ช่วยอธิบายแนวคิดของการลบเช็คและอัปเดตข้อมูลย้อนหลัง และเขียนอย่างไรไม่ให้เกิดภาวะคอขวดบนฐานข้อมูลขนาดใหญ่?

คำตอบของผู้สมัครที่คาดหวัง:

- ****แนวคิด:**** เมื่อแอปหลังบ้านเริ่มทำงาน (Startup) ตัวรันเนอร์ของ Seeder จะดึงข้อมูลยานพาหนะทั้งหมดมาเปรียบเทียบ หากพบว่าแบรนด์รถยนต์ยังไม่ใช่ยี่ห้อ "Isuzu" และรุ่นเป็นยี่ห้ออื่น จะนำไปแมปคู่เทียบ เช่น Camry → D-Max จากนั้นสั่งบันทึกอัปเดทย้อนกลับลง DB
- ****การป้องกันคอขวดบนระบบขนาดใหญ่:**** 1) ห้ามดึงข้อมูลมาทั้งหมดรวดเดียว: ไม่ควรใช้ `findAll()` กับฐานข้อมูลที่มีตารางขนาดใหญ่ แต่ควรใช้ Query เฉพาะที่มีเงื่อนไข `WHERE brand != 'Isuzu'` เพื่อกรองเอาเฉพาะแถวที่ต้องแก้ไขเท่านั้น 2) **การทำงานแบบกลุ่ม (Pagination & Batch Update):** แบ่งดึงข้อมูลออกเป็นรอบๆ รอบละ 500-1000 รายการ แล้วใช้ Hibernate Batch Processing อัปเดตพร้อมกันทีเดียว เพื่อป้องกันการใช้นหน่วยความจำเกินขนาด (Memory leak / Out of Memory) ในฝั่ง Java Application

Q12: ใน `application.yml` มีการตั้งค่า `spring.jpa.open-in-view: false` การกระทำนี้ช่วยเพิ่มความเร็วและเสถียรภาพต่อตัวจัดการ Connection Pool ของระบบได้อย่างไร และส่งผลกระทบต่อการใช้การโหลดข้อมูลสัมพันธ์แบบ Lazy Loading?

คำตอบของผู้สมัครที่คาดหวัง:

- ****ความเสถียรภาพ:**** ปิดการเชื่อมต่อฐานข้อมูลทันทีเมื่อทำงานผ่าน Controller เสร็จสิ้น ช่วยแก้ปัญหาการถือครอง Database Connection ไว้นานเกินไป (Connection Pool Starvation) ทำให้ฐานข้อมูลทำงานได้ราบรื่นขึ้น
- ****ผลกระทบ:**** หากพยายามเข้าถึงความสัมพันธ์ของ Entity ที่เป็น `@ManyToOne(fetch = FetchType.LAZY)` นอกกรอบของ Layer Service ที่ไม่มี Transaction อยู่ จะเกิดข้อผิดพลาด `LazyInitializationException` ทันที
- ****แนวทางการแก้ไข:**** ต้องทำการดึงข้อมูลลูกข่ายร่วมด้วยตั้งแต่ใน Service Layer โดยใช้ JPA Query ****`JOIN FETCH`**** หรือการเขียน Query ด้วย Entity Graph เพื่อโหลดข้อมูลความสัมพันธ์ทั้งหมดให้พร้อมตั้งแต่แรก (Eager Loading อย่างเหมาะสม)